



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 13 – Blockchain Application Design



Blockchain Application Design



- 1. From Smart Contract to Blockchain System**
- 2. Common DApp Architecture Patterns**
- 3. Backend Design in Blockchain Systems**
- 4. Interaction & Data Flow Models**
- 5. Exercises**

1. From Smart Contract to Blockchain System

Smart Contract vs Blockchain Application

	Smart Contract	Blockchain Application
Core Focus	On-chain logic	Complete system
Components	Solidity code	Contract + Frontend + Backend
Nature	Immutable	Hybrid architecture
Scope	Limited (gas-constrained)	Integrated with Web & Services (e.g., APIs, databases)

- ❖ A smart contract is a component
- ❖ A blockchain application is a distributed system.

Smart Contract → Blockchain Layer

Frontend + Backend + Database → Application Layer

1. From Smart Contract to Blockchain System

Typical DApp Layered Architecture

Blockchain Layer

- └ Ethereum, smart contracts, tokens
(immutable on-chain foundation)

Interaction Layer

- └ Wallets (MetaMask), RPC (JSON-RPC), APIs
(bridge to access the blockchain)

Application Layer

- └ Web frontend, backend services
(user-facing dApp logic)

Integration Layer

- └ Databases, external APIs, automation (event listeners)
(off-chain optimization & extensions)

2. Common DApp Architecture Patterns

Pattern A: User signs transactions

User → MetaMask → Smart Contract → Blockchain

- ❖ Decentralized
- ❖ User pays gas
- ❖ No backend signing
- ❖ Simple architecture

Typical use cases:

- ❖ DeFi
- ❖ NFT minting
- ❖ Voting DApps

2. Common DApp Architecture Patterns

Pattern B: Backend signs transactions

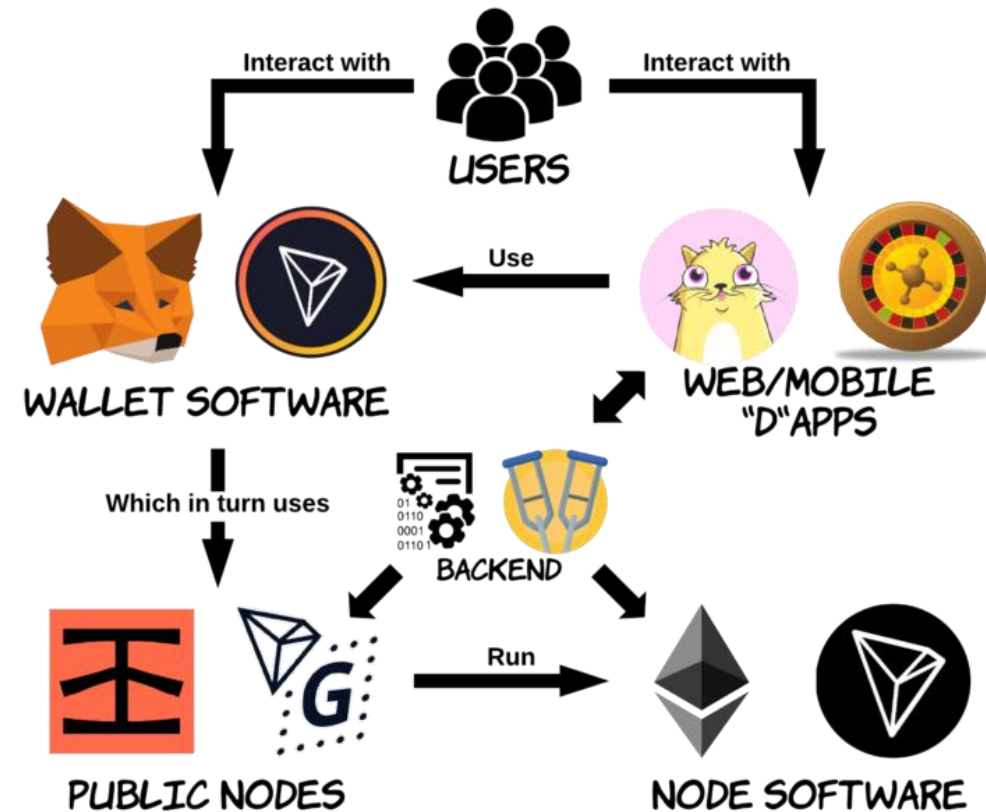
User → Frontend → Backend Server → Smart Contract → Blockchain

Backend responsibilities:

- ❖ Validate business logic
- ❖ Sign transactions using admin key
- ❖ Control permissions

Typical use cases:

- ❖ Enterprise blockchain
- ❖ Game rewards
- ❖ Platform-managed tokens
- ❖ Admin-controlled systems



⇒ Not all blockchain transactions are signed by users.

2. Common DApp Architecture Patterns

Pattern C: Hybrid Signing

- ❖ Users sign critical actions
- ❖ Backend signs administrative or automated actions

Example:

DAO voting → user signs

Reward distribution → backend signs

2. Common DApp Architecture Patterns

Architecture Comparison

	User-Signed Model	Server-Signed Model
Who signs transaction?	User wallet	Backend server
Gas payer	User	Platform
Decentralization	High	Medium
Control	Low	High
Complexity	Low	High
Security Risk	Wallet Compromise	Server Key Compromise

Examples:

User-Signed: Uniswap (users approve swaps via MetaMask).

Server-Signed: Centralized exchanges like Coinbase (backend handles on-chain settlements)

Q: When should we use backend signing?

3. Backend Design in Blockchain Systems

Smart contracts limitation:

- ❖ High gas cost for complex logic
- ❖ Limited computation and storage
- ❖ Poor support for complex queries
- ❖ No native scheduling or automation
- ❖ No direct access to external systems

Backend:

- ❖ Act as transaction orchestrator
- ❖ Act as admin account
- ❖ Automate contract interactions
- ❖ Implement off-chain logic

3. Backend Design in Blockchain Systems

When frontend queries blockchain directly:

- ❖ RPC calls are slow
- ❖ Rate limits exist
- ❖ Historical data is expensive
- ❖ Complex queries are impossible

Blockchain nodes are not designed for analytics or search.

Example:

Get all transfers of a user

Get transaction history

3. Backend Design in Blockchain Systems

Backend Roles in Blockchain Systems

- ❖ Transaction Signer
- ❖ Event Processor
- ❖ Business Logic Engine
- ❖ API Gateway
- ❖ Security Layer

Backend: blockchain controller, not just listener.

3. Backend Design in Blockchain Systems

Example: Backend Signing Transaction

```
import { ethers } from "ethers";

// Use environment variables for security
const provider = new ethers.JsonRpcProvider(process.env.RPC_URL);
// e.g., https://sepolia.infura.io/v3/YOUR_KEY

const ADMIN_PRIVATE_KEY = process.env.ADMIN_PRIVATE_KEY;

const wallet = new ethers.Wallet(adminPrivateKey, provider);

const CONTRACT_ADDRESS = "YOUR_CONTRACT_ADDRESS";
const CONTRACT_ABI = ["function mint(address to, uint amount)"];

const contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, wallet);

async function mintToken(userAddress) {
  const tx = await contract.mint(userAddress, 100);
  await tx.wait();
  console.log("Token minted for:", userAddress);
}
```

3. Backend Design in Blockchain Systems

Security Implications of Backend Signing

Risks:

- ❖ Private key leakage
- ❖ Centralization
- ❖ Abuse of admin power

Engineering solutions:

- ❖ Hardware wallets
- ❖ Multisig wallets
- ❖ Role-based contracts
- ❖ Limited permissions

⇒ Backend signing is powerful but dangerous.

4. Interaction & Data Flow Models

Interaction Models

Model 1: Direct User Interaction

User → Contract

e.g., Simple NFT minting via MetaMask

Model 2: User + Backend Coordination

User → Backend → Contract

e.g., User requests a reward, backend validates and signs.

Model 3: Event-Driven Automation

Contract → Event → Backend → Contract

4. Interaction & Data Flow Models

Event-Driven Architecture in Blockchain

- 1) Contract emits event
- 2) Backend detects event
- 3) Backend triggers new transaction
- 4) System state changes

Example:

A user sends ETH to a smart contract.

The smart contract emits an event: PaymentReceived.

The backend server listens to this event.

When the backend detects the event, it automatically sends reward tokens to the user.

⇒ Blockchain systems can be reactive systems.

4. Interaction & Data Flow Models

Engineering principle:

- ❖ Put **trust-critical** logic on-chain, **flexible** logic off-chain.
- ❖ Minimize on-chain logic to **reduce costs**
- ❖ Use off-chain for **scalability**

Logic Type	On-chain	Off-chain
Ownership	Yes	No
Token balance	Yes	No
Business rules	Yes but limit	Yes
UI logic	No	Yes
Data analytics	No	Yes
Admin automation	No	Yes

5. Exercises

Ex 1: Build an app where users and backend both send transactions.

Smart Contract:

- ❖ `registerUser()` → called by users (MetaMask)
- ❖ `rewardUser(address)` → called by admin (backend)
- ❖ `getReward(address)` → view reward balance

Emit events when users register and receive rewards

Frontend:

- ❖ Connect MetaMask
- ❖ Register user
- ❖ Show reward balance

Backend:

- ❖ Use admin private key
- ❖ Call `rewardUser()` for users

5. Exercises

Ex 2: Build a system to view blockchain events using backend instead of direct blockchain query or live event update.

Smart Contract:

- ❖ `submitMessage(string)`
- ❖ Emit event `MessageSubmitted(address, string)`

Backend:

- ❖ Listen to `MessageSubmitted` events
- ❖ Store events

Frontend:

- ❖ Submit message to blockchain
- ❖ Show event history from backend